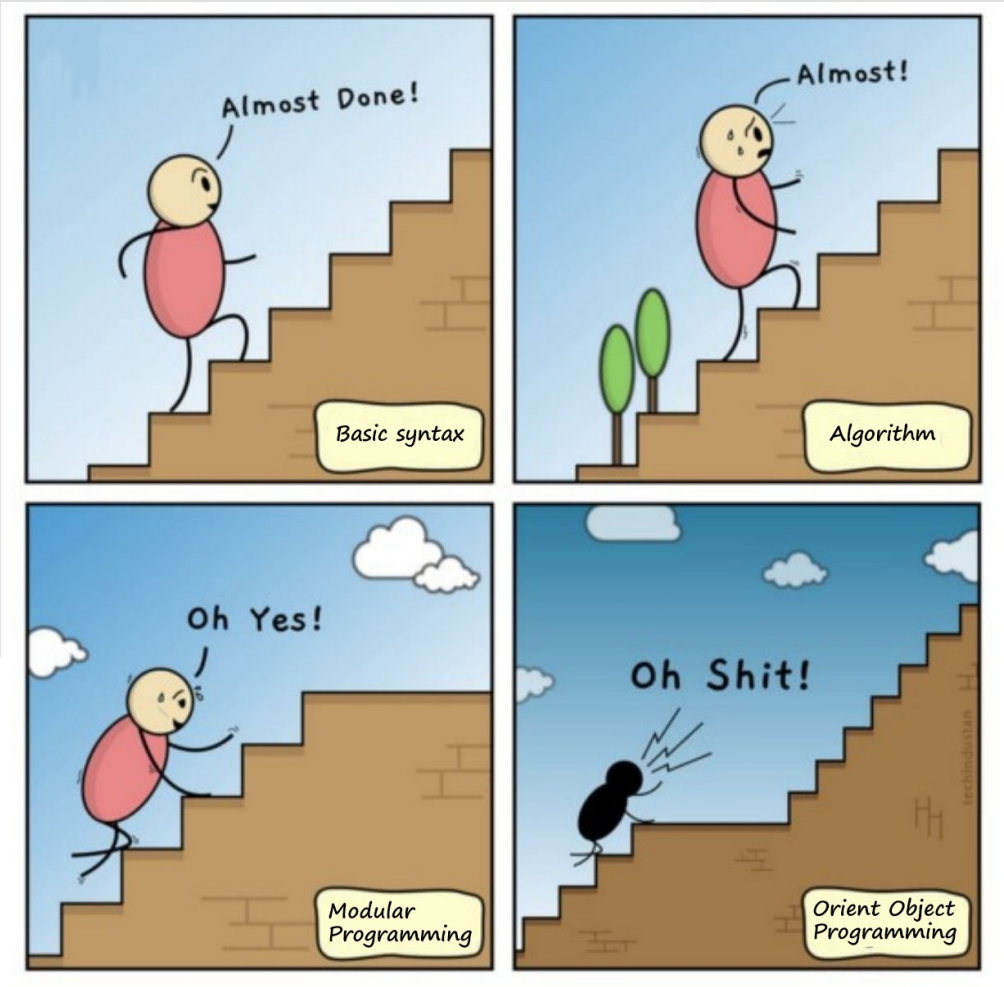
Tema 04. Programación orientada a objetos



Índice

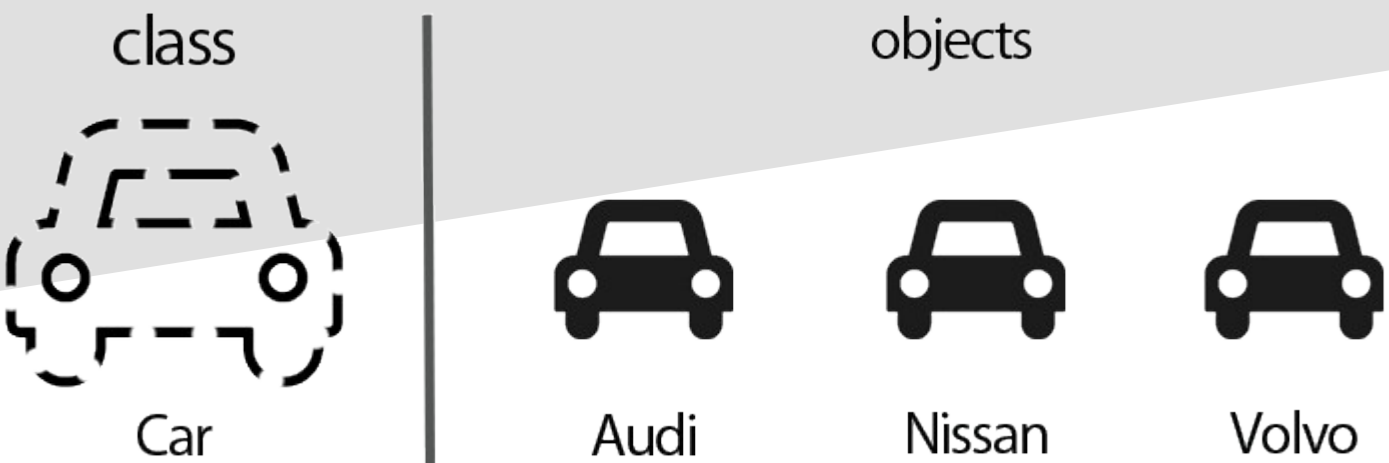
1. Introducción.
2. Características de la P.O.O.
3. Declaración de una clase.
4. Constructores.
5. Objetos.
6. Métodos de clase.
7. Elementos estáticos.
8. Convenciones utilizadas en Java.

1.- Introducción.

Java es un lenguaje orientado a **objetos**.

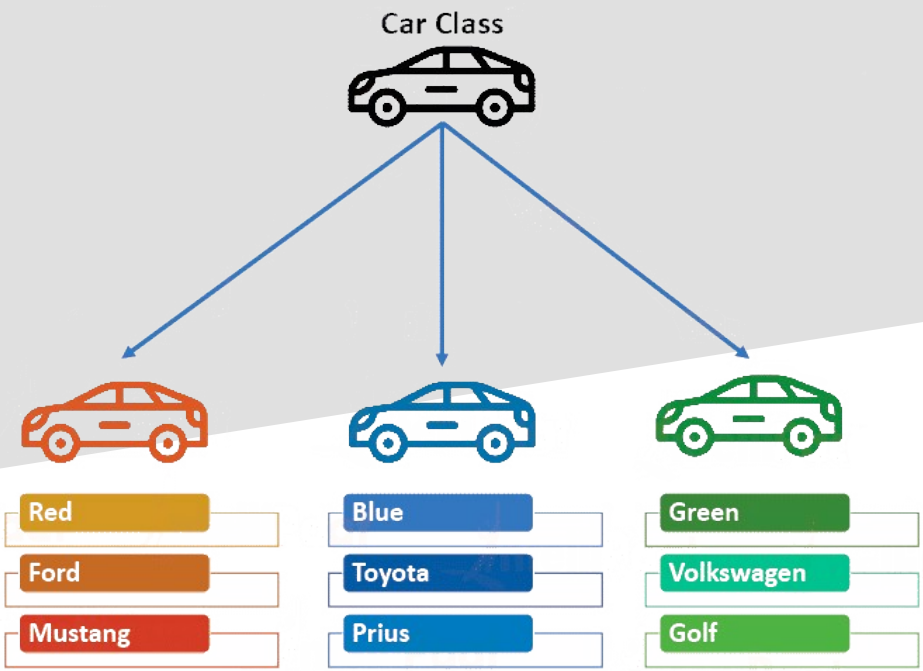
Toda aplicación Java está formada por un conjunto de **clases.**

Hay que tener claros los conceptos: Clase y Objeto



1.- Introducción.

- **Clase**: abstracción de un concepto que posee datos y funcionalidad.
- **Objeto**: caso concreto (instancia) de una clase.



Una clase es un molde con el cual se generan los objetos: define sus propiedades y su comportamiento.

1.- Introducción.

Componentes de una clase

- **Atributos**: son las propiedades de los objetos de la clase.
- **Constructores:** procedimientos que se ejecutan en el momento de crear un objeto clase. Tienen el mismo nombre de la clase y pueden existir varios.
- **Métodos**: procedimiento u operaciones que comparten los objetos de clase.

Nombre_clase
Atributos: datos comunes
Constructores: crean objetos de la clase
Métodos: modifican / consultan datos del objeto / clase

1.- Introducción.

Diagramas de clases

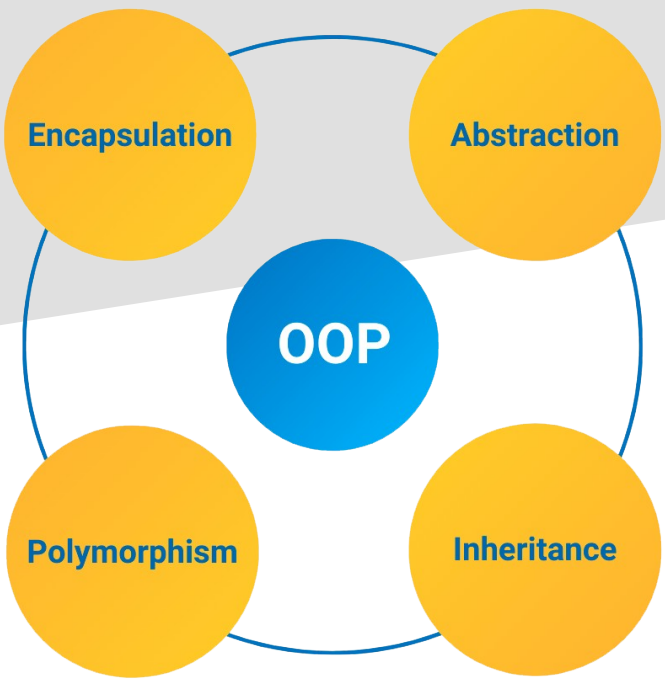
Coche
Atributos: color marca modelo velocidad ...
Constructores: Coche() ...
Métodos: arrancar parar girar ...

Cuenta
Atributos: numCuenta saldo titular banco ...
Constructores: Cuenta() ...
Métodos: sacarDinero ingresarDinero consultarSaldo ...

2.- Características de la P.O.O.

La programación orientada a objetos se basa en 4 conceptos:

- Abstracción.
- Encapsulación.
- Herencia.
- Polimorfismo.



2.- Características de la P.O.O.

ABSTRACCIÓN

Un objeto es capaz de desempeñar una función independientemente del contexto en que éste es utilizado. Es decir, en cualquier ámbito un objeto tiene las mismas propiedades y se comporta de la misma forma.

Se habla de abstracción en dos sentidos:

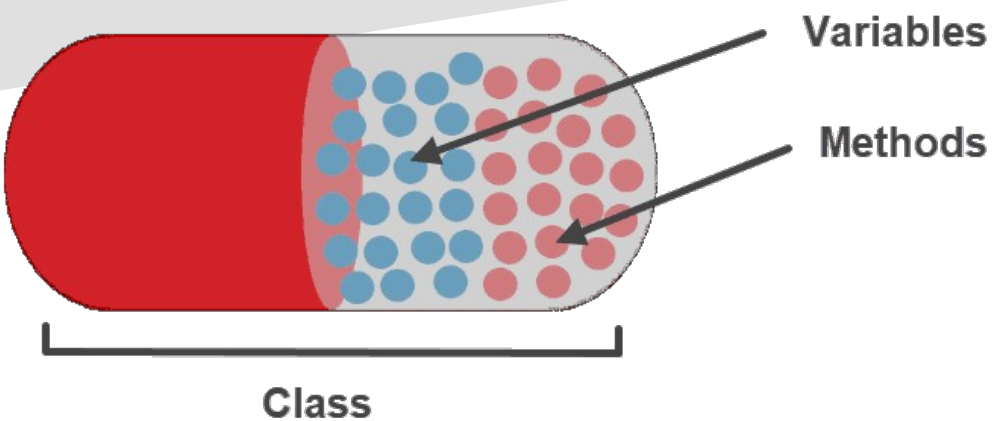
- **Funcional**: conociendo los métodos que tiene una clase, podremos usar el objeto sin conocer como funcionan internamente. Por ejemplo: `Math.sqrt` calcula la raíz cuadrada pero no sabemos como (ni nos importa).
- **De datos**: manera en que se almacenan o definen los atributos, también es irrelevante para el diseño del objeto. Por ejemplo: el color puede definirse como la palabra `red`, o como un vector RGB `(255,0,0)`.

2.- Características de la P.O.O.

ENCAPSULACIÓN

La encapsulación está íntimamente relacionada con la ocultación de la información, definiendo qué partes de un objeto son visibles (`public`) y qué partes son ocultas (`private`).

El programador que usa una clase ya creada no tiene por qué saber como funciona internamente los métodos de la clase.



2.- Características de la P.O.O.

HERENCIA

La herencia permite definir a partir de una clase, otras clases relacionadas.

Esto supone:

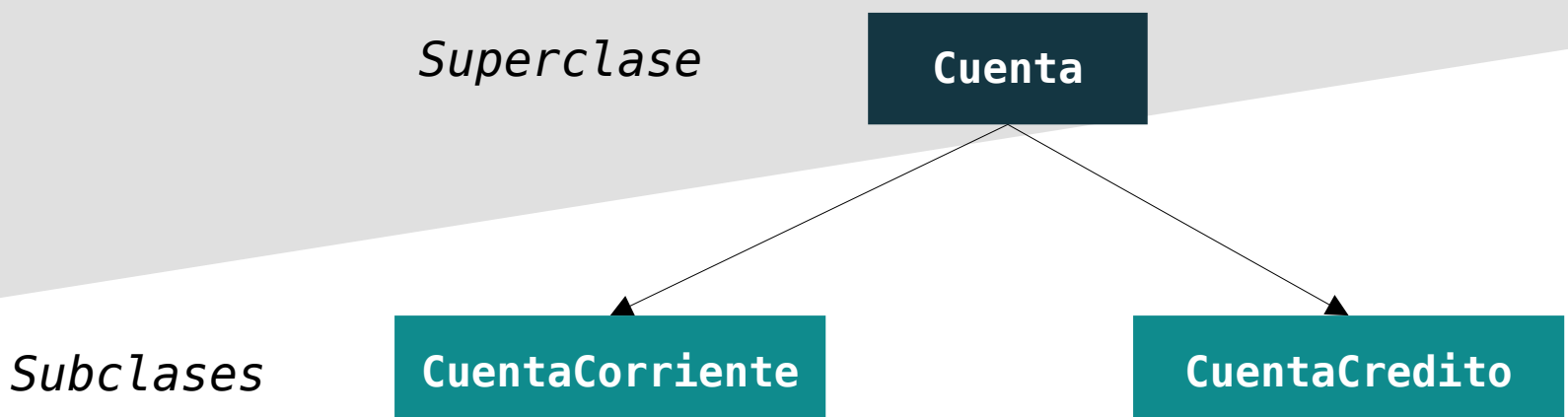
- **Especialización** de la clase dada. Por ejemplo: la clase `CuentaCorriente` es un especialización de la clase `Cuenta`.
- **Generalización** de la clase dada. Por ejemplo: la clase `Cuenta` es una generalización de la clase `CuentaCorriente`.

*Ventaja: no es necesario repetir código, basta con decir que una clase **extiende** a la otra o **hereda** de ella.*

2.- Características de la P.O.O.

HERENCIA

La clase `Cuenta` es la clase padre (o superclase). Sus métodos y atributos son heredados por las clases derivadas (hijas o subclases).

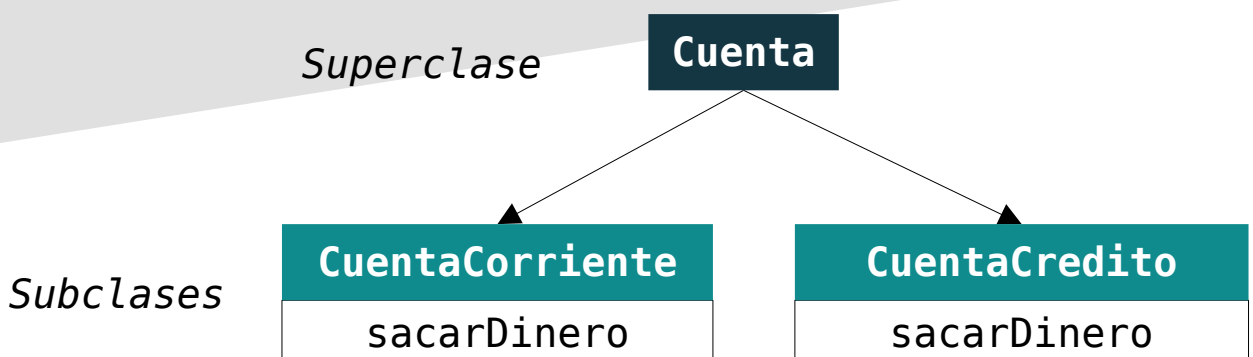


2.- Características de la P.O.O.

POLIMORFISMO

El polimorfismo permite que objetos de un mismo super(tipo) puedan realizar una operación distinta.

Por ejemplo: la clase `Cuenta` tiene dos subclases `CuentaCorriente` y `CuentaCredito`. El método `sacarDinero` funciona de forma diferente en una cuenta corriente y en una de crédito.



3.- Declaración de una clase.

La forma básica para declarar una clase en Java es:

```
[modificadorAcceso] class NombreClase {
    // Atributos o propiedades de la clase
    // (color, velocidad, marca...)
    // ...

    // Constructor o constructores
    // ...

    // Metodos de la clase
    // arrancar, parar, acelerar...
    // ...
}
```

3.- Declaración de una clase.

MODIFICADORES DE ACCESO DE CLASE

Son palabras reservadas que se anteponen a la declaración de la clase e indican la visibilidad de la clase.

public	Accesible para todas las clases
[nada]	Si no se indica nada, la clase será accesible sólo desde clases que estén en el mismo paquete (friendly).

3.- Declaración de una clase.

3.1 Atributos

Los atributos de una clase se definen según esta sintaxis:

```
[modAcceso] [modiAtributo] tipo nombre [= valorInicial]
```

Donde "`nombre`" es el nombre que daremos al atributo, siendo un nombre válido según las normas del lenguaje: por convención, en Java, los nombres de los atributos empiezan con una letra minúscula.

Los datos en naranja representan palabras reservadas del lenguaje Java.

Los datos entre [ ] son opcionales.

<u>MODIFICADORES DE ATRIBUTOS</u>: son características específicas del atributo.

Los posibles valores son: `static` o `final` (lo veremos más adelante)

3.- Declaración de una clase.

3.1 Atributos

MODIFICADORES DE ACCESO DE MÉTODOS / ATRIBUTOS

Indica desde que parte del código se puede acceder a la variable.

public	El método o atributo es siempre visible
private	El método o atributo es privado. Sólo es visible desde dentro de la propia clase.
protected	El método o atributo es visible sólo por la propia clase y las que hereden de ella.
[nada]	El método o atributo es visible pero sólo desde las clases que se encuentren en el mismo paquete (friendly).

3.- Declaración de una clase.

3.2 Métodos

- Es un bloque de código que realiza una tarea específica.
- Un método pertenece a una clase, y podrá acceder a los atributos de la misma (salvo que sea estático).
- Un método tiene:
  - Una lista de parámetros de entrada al método (datos que necesita el método para hacer la tarea específica).
  - Un valor devuelto.
  - Puede tener también variables locales al método.
- Un método puede no tener ningún parámetro y/o no devolver ningún valor.
- Pueden ser llamado o invocado desde cualquier punto de la aplicación.

3.- Declaración de una clase.

3.2 Métodos

Para definir los métodos se emplea la siguiente sintaxis:

```
[modAcceso][modMetodo]tipoRetorno nombreMetodo (listaParametros)
{
   //variables locales al metodo
   //codigo del metodo
   return expresion; // cuando el tipoReorno es void, no se pone
}
```

modAcceso: `public`, `private`, `protected`, (`friendly`), igual que los atributos.

modMétodo: `abstract`, `static` o `final` (ya veremos que significan).

tipoRetorno: tipo de valor devuelto por el método (`void` si no devuelve valor).

3.- Declaración de una clase.

3.2 Métodos

listaParametros:

El método o función puede tener una lista de argumentos o parámetros. Estos parámetros estarán separados por comas y definidos como:

```
tipo nombreParámetro
```

Los argumentos de los tipos básicos o primitivos se pasan siempre **por valor**. El método recibe una copia del argumento actual; si se modifica esta copia, el argumento original que se incluyó en la llamada no queda modificado.

Los objetos se pasan **por referencia**, es decir, a través de las referencias se pueden modificar los objetos referenciados.

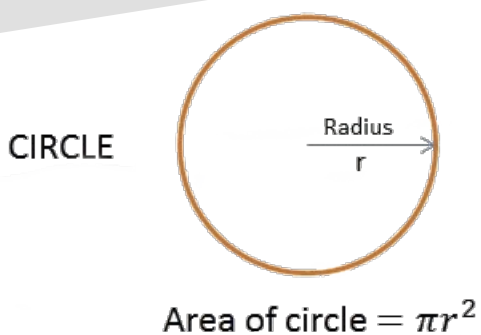
3.- Declaración de una clase.

3.2 Métodos

tipoRetorno:

Si el método tiene algún tipo de retorno, quiere decir que ha de devolver un valor de dicho tipo. Esto se hace mediante la palabra reservada **`return`**.

Ejemplo: un método de la clase `Circulo` que calcula el área de un círculo, debería devolver el dato área (`double`) que ha calculado.



3.- Declaración de una clase.

Ejemplo:

class Circulo

```
public class Circulo {
    // Atributos de la clase
    private double radio;
    public final double PI = 3.14;

    // Aquí falta el método constructor

    // Métodos de la clase
    public double calculaArea() {
        return PI * Math.pow(radio, 2);
    }

    public void setRadio(double radio) {
        if (radio > 0)
            this.radio = radio;
    }

    public double getRadio() {
        return this.radio;
    }
}
```

4.- Constructores.

Cuando se crea o instancia un objeto de una clase es necesario dar un valor inicial a sus atributos, es por ello que existe un método especial en cada clase, llamado **constructor**.

El constructor es llamado automáticamente al crear un objeto de la clase.

Para definir los constructores se emplea la siguiente sintaxis:

```
nombreConstructor  (listaParametros)
```

`nombreConstructor` debe coincidir con el nombre de la clase.

4.- Constructores.

Los constructores poseen unas características especiales:

- Se tienen que llamar igual que la clase.
- No tienen valor de retorno (ni siquiera void).
- Pueden existir varios, que se distinguirán por los parámetros que aceptan (sobrecarga).
- Debe existir al menos 1 constructor.
- Si no se codifica ningún constructor, se crea uno **por defecto** sin argumentos.
- Si la clase tiene algún constructor, el constructor por defecto no se crea.

4.- Constructores.

Ejemplo: `clase Cuenta`

```java
public class Cuenta {
// Atributos de la clase
    private double saldo;

    /*
     * Primer Método constructor. Si no se indica el saldo se considera saldo 0
     */
    public Cuenta() {
        saldo = 0;
    }

    /*
     * Otro Método constructor. El objeto Cuenta se crea con un saldo inicial
     */
    public Cuenta(double saldoinicial) {
        saldo = saldoinicial;
    }

    public void ingresarCantidad(double cantidad) {
        // .....
    }
}
```

4.- Constructores.

Ejemplo:

clase Circulo

```
public class Circulo {
    // Atributos de la clase Círculo
    private double radio;
    public final double PI = 3.14;

    // Método constructor
    public Circulo(double radioInicial) {
        radio = radioInicial;
    }

    // Métodos de la clase Círculo
    public double calcularArea() {
        double area;
        area = PI * Math.pow(radio, 2);
        return area;
    }
//....
}
```

5.- Objetos.

Un objeto es una instancia de una clase.

En nuestros programas tendremos objetos que ejecutan métodos.

- Para poder utilizar un objeto hay que:
  - Declarar una referencia a la clase.
  - Crear un objeto mediante el operador new, invocando al constructor adecuado.
- Para invocar un método de un objeto:

```
nombreObjeto.nombreMetodo(…)
```

5.- Objetos.

Ejemplo: `clase TrabajandoConCuentas`

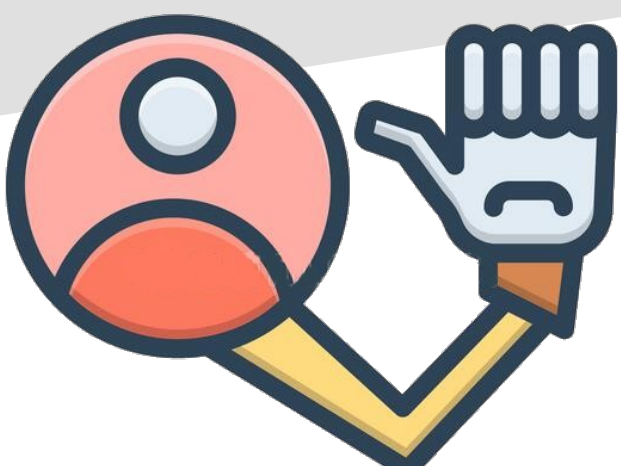
```
public class TrabajandoConCuentas {

    public static void main(String[] args) {
        Cuenta cuenta1; // declarar la la referencia al objeto cuenta1
        cuenta1 = new Cuenta(); // crear el objeto cuenta1.
        //LLamará a constructor de la clase cuenta

        Cuenta cuenta2 = new Cuenta();
        // en un solo paso hace lo mismo con una nueva referencia cuenta2

        cuenta1.ingresarCantidad(100);
        cuenta2.ingresarCantidad(200);
        cuenta1.sacarCantidad(50);
    }
}
```

5.- Objetos.

THIS

Java proporciona una referencia al objeto con el que se está trabajando. Esta referencia se denomina **`this`**, que es el objeto que está ejecutando el método. En los ejemplos que hemos visto hasta ahora, se obviaba esta referencia, puesto que se sobreentiende que nos referimos al objeto que está invocando al método.

En algunas ocasiones nos servirá para resolver ambigüedades o para devolver referencias al propio objeto.



5.- Objetos.

Ejemplo: `this`

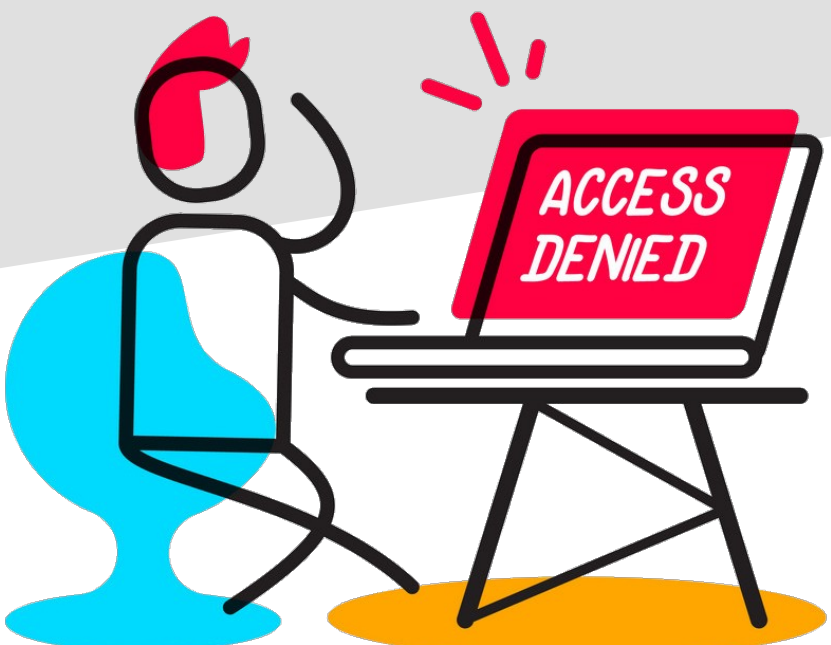
```
public class Circulo {
    // Atributos de la clase Círculo
    private double radio;
    public final double PI = 3.14;

    // Método constructor
    Circulo(double radio) {
        this.radio = radio; //Resuelve ambigüedad
    }
}
```

6.- Métodos de clase.

Los **sets** y **gets** son la forma de acceder a los atributos de una clase.

Generalmente, se usan con los atributos privados, ya que a los públicos se puede acceder directamente sin tener que acudir a ellos.

Por uno de los principios de la POO, los atributos de los objetos deben ser siempre "privados" (concepto "encapsulación": no son accesibles desde fuera del objeto, solo el objeto tiene la potestad de usarlos directamente) y se deberán crear métodos públicos para obtener su valor.



6.- Métodos de clase.

- Método "**setter**" para asignar un valor a una variable. La sintaxis es:

```
public void setNombreAtributo(TipoPropiedad valor) {    }

public void setNombre(String nombre) {
    this.nombre = nombre;
}
```

- Método "**getter**" para retornar el valor (devolver la información del atributo). La sintaxis es:

```
public TipoPropiedad getNombreAtributo() {
    return tipoPropiedad;
}

public String getNombre() {
    return this.nombre;
}
```

6.- Métodos de clase.

Ejemplo:

“getters”

“setters”

```
public class Persona {
    private String nombre;
    private int edad;

    public Persona() { // Constructor
    }

    public void setNombre(String nombre) { // Asignar un valor
        this.nombre = nombre;
    }

    public String getNombre() { // Devolver el valor
        return this.nombre;
    }

    public void setEdad(int edad) {
        if (edad >= 0)
            this.edad = edad;
    }

    public int getEdad() {
        return this.edad;
    }
```

6.- Métodos de clase.

Los métodos “**getters**” y “**setters**” lo único que hacen es proporcionar acceso a unos atributos que son privados y que no serían accesibles desde otras clases. Además los set controlarán que no se den a los atributos valores no válidos.

Ejemplo:

```
Persona p = new Persona();
p.setNombre("Pepe");
p.setEdad(23);
System.out.println("Su nombre es: " + p.getNombre());
System.out.println("Su edad es: " + p.getEdad());
```

6.- Métodos de clase.

El método **`toString`** es un método que tienen todos los objetos Java (porque todos heredan de `Object`).

- Devuelve un `String` con la información de los atributos del objeto.

Cuando un objeto se imprime con

```
System.out.println ("El objeto contiene" + o);
```

se está llamando a `o.toString()` es decir, sería equivalente a

```
System.out.println("El objeto contiene" + o.toString() );
```

Podemos redefinirlo para que devuelva la información del objeto que nosotros queramos.

6.- Métodos de clase.

En Java si se comparan dos objetos con el operador == se están comparando las referencias, no el contenido del objeto. Por defecto, cuando creamos una nueva clase, ya tiene un método **`equals`**, pero compara las referencias.

Debemos redefinir el método **`equals`** para comparar correctamente objetos de esa clase.

El método **`hashCode`** complementa a **`equals`**. Existe para mejorar el rendimiento de Java a la hora de comparar dos objetos.

En eclipse para generar estos métodos iremos a “Source > Generate hashCode y equals”. Elegiremos los campos que deben coincidir para que los objetos se consideren iguales.

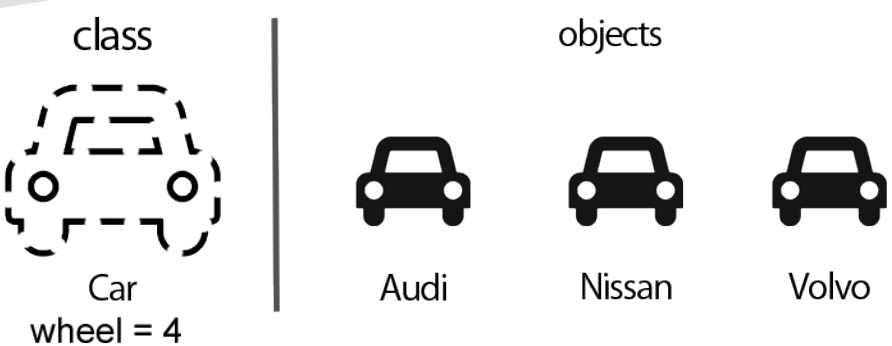
7.- Elementos estáticos.

ATRIBUTOS

Una clase puede tener variables propias de la clase y no de cada objeto. A estas variables se les llama variables de clase o variables `static` y se suelen utilizar para definir constantes comunes para todos los objetos de la clase o variables que sólo tienen sentido para toda la clase.

Las variables `static` son compartidas por todos los objetos de la clase (no se crea una copia por objeto, como con las variables de instancia).

El acceso a las variables estáticas se puede realizar desde la clase o desde los objetos.



7.- Elementos estáticos.

MÉTODOS

Para llamar a un método estático no hace falta crear un objeto de la clase en la que se define:

- Si se invoca desde la clase en la que se encuentra definido, basta con escribir su nombre.
- Si se le invoca desde una clase distinta, debe anteponer a su nombre, el de la clase en la que se encuentra seguido del operador punto (.)

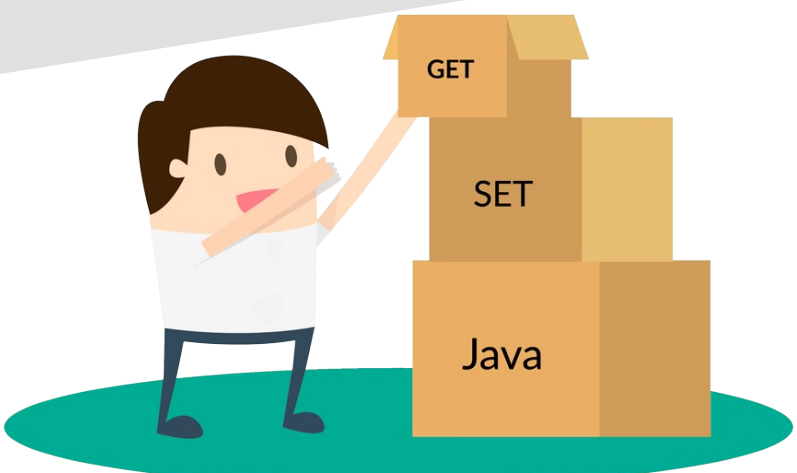
```
<NombreClase>.metodoEstatico()
```

```
double aleatorio = Math.random();
```

7.- Elementos estáticos.

RESTRICCIONES MÉTODOS ESTÁTICOS

- Solo pueden acceder a variables y métodos estáticos de la misma clase.
- No puede acceder a variables y/o métodos no estáticos directamente, tiene que crear primero un objeto.
- Los métodos de instancia sí pueden acceder a variables y métodos estáticos.
- Un constructor no puede ser estático.
- No pueden acceder a `this`.



7.- Elementos estáticos.

VARIABLES Y MÉTODOS ESTÁTICOS CONOCIDOS

- Variables estáticas:

```
Integer.MAX_VALUE;
Math.PI;
//...
```

- Métodos estáticos: en la clase Math, todos los métodos son static:

```
public static double pow(double x, double y)
```

y se llaman usando el nombre de la clase:

```
Math.pow(2, 3);
```

7.- Elementos estáticos.

Una variable **`final`** no puede cambiar su valor a lo largo de la ejecución del programa, puede ser considerada como una constante.

Deben ser inicializadas en su declaración.

Por convenio, el identificador de una variable `final`, se escribe en mayúsculas.

Es lógico definir las constantes como `static`, de forma que sean compartidas por todos los métodos y objetos que se creen.

La sintaxis es:

```
final [static] tipo NOMBRE = valor;
```

```
final static double PI = 3.141592;
```

8.- Composición de clases.

Los atributos de una clase no se limitan a tipos básicos de Java. Un atributo puede ser de tipo objeto, definido en librerias o por nosotros, como una clase más.

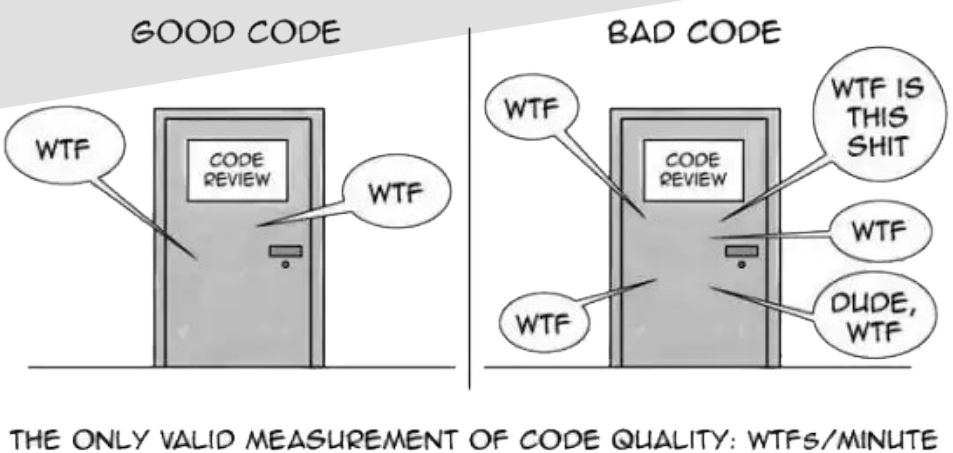
Ejemplo:

```java
public class Cliente {
    // Atributos de la clase
    String dni;
    String nombre;
    String apellidos;

    int telefono;
}
```

```java
public class Cuenta {
    // Atributos de la clase
    double saldo;
    String numCuenta;

    //Atributo Objeto Cliente
    Cliente cliente;

}
```

9.- Convenciones utilizadas en Java.

- Los nombres de las clases comienzan por mayúscula.
- Los nombres de variables, métodos y objetos comienzan por minúscula.
- Si contienen varias palabras se unen evitando subrayado y separando palabras con mayúscula (CamelCase).
- Los nombres de las constantes son en mayúscula.
- El código se tabula para mejorar su comprensión.



Tema 04. Programación orientada a objetos